



# HACKATÓN BY PASEO · 3 DÍAS ·

# Menú de contenido

Haga clic en una sección para ir a ella; cada lámina tiene un botón para volver aquí.

**01**

## **El reto**

Resumen del reto · Por qué este caso

**02**

## **El caso de uso**

Problema y usuarios · Los 4 agentes · Recorrido del usuario

**03**

## **Alcance y datos**

Alcance del MVP · Datos del reto · Escenarios sembrados · Kit de insumos

**04**

## **Solución técnica**

Arquitectura · Stack · Backend · API y capa semántica · Frontend y UX · Pantallas · IA responsable

**05**

## **Metodología y agenda**

Roles del equipo · Construcción por capas · Agenda · Reglas y definición de terminado

**06**

## **Evaluación**

Entregables · Criterios · Pruebas del jurado

**07**

## **Comercialización**

Hoja de ruta · Modelo de negocio y PI · Fuentes



# 01

## El reto

# En 3 días, cada equipo construye Centinela

[← Menú](#)

Un sistema de agentes de IA que **vigila los datos del negocio**, detecta problemas antes de que cuesten dinero, explica por qué pasan y propone (o ejecuta, con aprobación humana) la acción para corregirlos.

Es el paso de los tableros que alguien tiene que mirar a una operación que se vigila sola. Todos los equipos usan el mismo dataset sintético, con problemas sembrados que el jurado usa para probar cada solución.

3

días presenciales en la sede de On Business

4–5

personas por equipo: negocio, datos, backend y frontend

6

escenarios por descubrir: 5 anunciados y 1 oculto

## Resultado esperado



MVP funcionando



Demo de 5 minutos



Pitch de negocio de 3 minutos



# 02

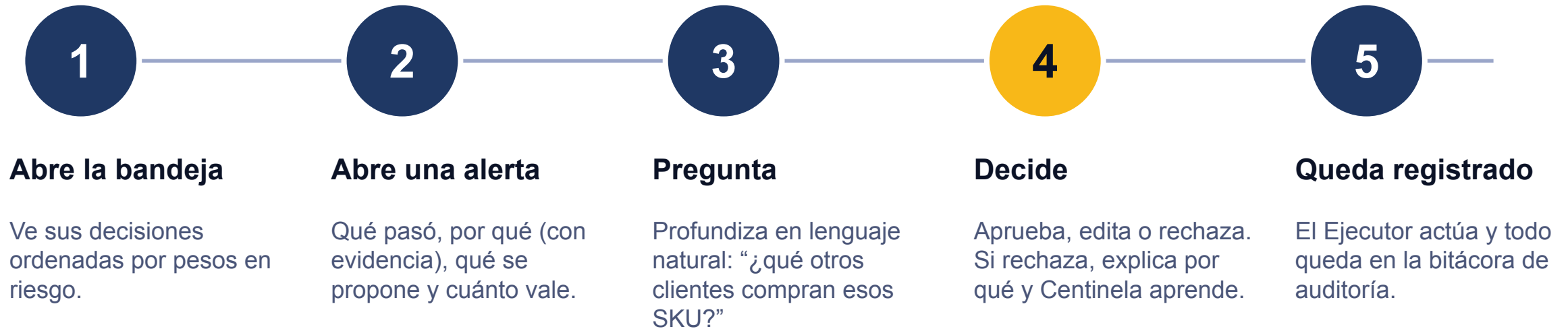
## El caso de uso

# Los 4 agentes: tres leen, solo uno actúa y únicamente tras aprobación

[< Menú](#)

**Orquestador:** coordina a los agentes, guarda el estado de cada alerta y detiene el flujo antes de toda acción con efecto externo.

# El recorrido del usuario: del aviso a la decisión en minutos

[← Menú](#)

**Principio:** el usuario no busca el problema; el problema llega a él, explicado y con una propuesta lista para aprobar.



# 03

## Alcance y datos

Qué debe funcionar y con qué datos se prueba

# Alcance del MVP:

[← Menú](#)

## Base

Obligatorio

- Vigía detecta al menos 3 de los 5 escenarios
- Analista explica la causa con evidencia
- Bandeja con aprobar / rechazar
- Chat para preguntar sobre los datos
- Bitácora de cada decisión

**Fuera de alcance:** conexión a sistemas reales de clientes, autenticación empresarial completa y despliegue productivo (van en el pitch como hoja de ruta).

# Datos del reto: 12 meses de una distribuidora ficticia

[Menú](#)

**269.595**

filas en 15 tablas

**12**

meses: oct 2025 – sep 2026

**≈ \$23.500 M**

en ventas netas (COP)

**CSV + SQL**

PostgreSQL listo para usar

| Tabla                            | Contenido                             | Filas   |
|----------------------------------|---------------------------------------|---------|
| clientes                         | Segmento, ciudad, cupo, plazo         | 500     |
| vendedores                       | Fuerza de ventas por región           | 25      |
| productos                        | SKU, línea, proveedor, clase ABC      | 200     |
| proveedores                      | Lead time y país                      | 40      |
| lista_precios · costos_proveedor | Precios y costos con vigencias        | 2.804   |
| ordenes_compra                   | OC a proveedores, retrasos            | 3.802   |
| pedidos                          | Encabezado, canal y estado            | 20.013  |
| pedidos_detalle                  | Cantidad, precio, descuento, costo    | 60.103  |
| facturas · pagos                 | Cartera: vencimientos y pagos         | 36.095  |
| inventario_diario                | Existencias por SKU, bodega y día     | 146.000 |
| politicas/ (PDF)                 | Crédito, descuentos, inventario (RAG) | 3       |

# Kit de insumos: todo lo necesario para empezar en 5 minutos

[← Menú](#)

## Centinela\_Kit\_Equipos.zip

|                                                                                                                                |                              |
|--------------------------------------------------------------------------------------------------------------------------------|------------------------------|
|  <code>datos/csv/</code>                      | 15 tablas en CSV (UTF-8)     |
|  <code>datos/sql/01_esquema.sql</code>        | Tablas en PostgreSQL         |
|  <code>datos/sql/02_carga.sql</code>          | Carga de los CSV             |
|  <code>datos/sql/03_capa_semantica.sql</code> | 7 vistas de métricas         |
|  <code>datos/metricas.yaml</code>             | Definición única de KPIs     |
|  <code>politicas/</code>                      | 3 políticas en PDF           |
|  <code>diccionario_de_datos.xlsx</code>       | Campos, tipos y ejemplos     |
|  <code>evaluaciones/</code>                 | Plantilla de casos de prueba |
|  <code>generador/</code>                    | Datasets alternativos        |
|  <code>README.md</code>                     | Guía de inicio               |

## Puesta en marcha

```
createdb centinela
cd datos
psql -d centinela -f sql/01_esquema.sql
psql -d centinela -f sql/02_carga.sql
psql -d centinela -f sql/03_capa_semantica.sql
```



# 04

## Solución técnica

Arquitectura, stack, backend, frontend e IA responsable

# Arquitectura de referencia

[Menú](#)



# Stack recomendado (los equipos pueden cambiar piezas si lo justifican)

| Capa                          | Recomendado                                                       | Alternativas válidas                               |
|-------------------------------|-------------------------------------------------------------------|----------------------------------------------------|
| Modelos de lenguaje           | Claude: un modelo grande para razonar, uno rápido para clasificar | GPT, Gemini; modelos abiertos vía Ollama           |
| Orquestación de agentes       | LangGraph (estado y pausa para aprobación humana)                 | Claude Agent SDK, OpenAI Agents SDK, ADK, CrewAI   |
| Herramientas                  | Servidores MCP: SQL de solo lectura, políticas, acciones          | Function calling nativo del modelo                 |
| Datos                         | PostgreSQL + pgvector                                             | DuckDB para análisis local                         |
| Detección de anomalías        | Reglas de negocio + estadística (z-score, tendencia)              | scikit-learn (Isolation Forest), Prophet           |
| Backend                       | Python + FastAPI + Pydantic                                       | Node.js + Hono / NestJS                            |
| Frontend                      | Next.js + React + Tailwind + shadcn/ui + Recharts                 | Streamlit solo para prototipo (baja la nota de UX) |
| Observabilidad y evaluaciones | Langfuse + promptfoo                                              | LangSmith, Arize Phoenix, Ragas                    |
| Despliegue                    | Docker Compose; demo en Cloud Run, Render o Vercel                | Local con túnel seguro                             |

**Regla de oro:** los números los calcula SQL o Python; el modelo razona, explica y redacta, nunca inventa cifras.

# Backend: cifras correctas, acciones controladas y costo predecible

[← Menú](#)

## Capa semántica

KPIs definidos una vez como vistas SQL; los agentes no consultan tablas crudas.



## Reloj simulado

Endpoint para avanzar día a día: el jurado adelanta la operación y ve aparecer las alertas.



## Ciclo de vida

Nueva → en análisis → propuesta → aprobada o rechazada → ejecutada, con estado persistido.



## Aprobación humana

El flujo se detiene antes de cualquier acción externa; sin aprobación no hay acción.



## Acciones seguras

Lista cerrada de herramientas, idempotentes, en borrador o sandbox.



## Priorización

Alertas ordenadas por pesos en riesgo; una misma causa no genera 10 alertas.



## Costo y latencia

Modelo rápido para lo simple, caché, tope de tokens y costo registrado por alerta.



## Robustez

Reintentos, timeouts y “no tengo evidencia suficiente” como respuesta válida.

# API y capa semántica incluidas en el kit

[◀ Menú](#)

## API mínima

|             |                                                                              |
|-------------|------------------------------------------------------------------------------|
| <b>POST</b> | <code>/simulacion/avanzar?dias=1</code><br>mueve el reloj y dispara al Vigía |
| <b>GET</b>  | <code>/alertas?estado=propuesta</code><br>bandeja de decisiones              |
| <b>GET</b>  | <code>/alertas/{id}</code><br>causa, evidencia y acciones                    |
| <b>POST</b> | <code>/alertas/{id}/decision</code><br>aprobar, editar o rechazar + motivo   |
| <b>POST</b> | <code>/chat</code><br>preguntas en lenguaje natural (streaming)              |
| <b>GET</b>  | <code>/bitacora</code><br>auditoría de decisiones y acciones                 |

## Vistas de métricas (PostgreSQL)

### `v_ventas`

Ventas netas con costo y margen por línea de pedido

### `v_margen_semanal_linea`

Margen semanal por línea de producto

### `v_cartera_cliente`

Saldo abierto, vencido y días de pago por cliente

### `v_dias_pago_mensual`

Tendencia de días de pago por mes de factura

### `v_cobertura_inventario`

Cobertura en días y unidades pendientes por SKU y bodega

### `v_descuentos_fuera_politica`

Líneas sobre el tope y descuento en exceso

### `v_actividad_cliente`

Días sin comprar frente al intervalo habitual

`fecha_corte()` devuelve el último día del dataset; con el reloj simulado se reemplaza por el día simulado.

# Frontend y UX: una bandeja de decisiones, no un tablero

[← Menú](#)

Alta

Margen · línea Hogar

Confianza alta · hoy 7:00 a. m.

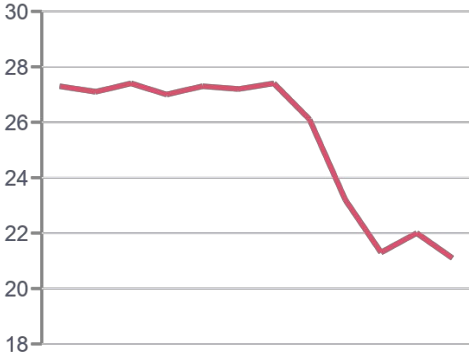
## El margen de la línea Hogar cayó 6 puntos en 3 semanas

\$42 M/mes recuperables

**POR QUÉ**  
El proveedor X subió el costo de 4 SKU y el precio de venta no se ajustó.

**PROPUESTA**  
Ajustar el precio 3% en esos 4 SKU y renegociar con el proveedor X.

Margen semanal (ilustrativo)



| Semana | Margen (ilustrativo) |
|--------|----------------------|
| 1      | 27.5                 |
| 2      | 27.2                 |
| 3      | 27.5                 |
| 4      | 27.0                 |
| 5      | 27.3                 |
| 6      | 27.4                 |
| 7      | 26.5                 |
| 8      | 23.5                 |
| 9      | 21.5                 |
| 10     | 22.0                 |
| 11     | 21.5                 |

Aprobar

Editar

Rechazar

Ver cómo llegué aquí ›

- ✓ **Valor en 30 segundos**  
Al entrar, el dinero en riesgo hoy y las 3 decisiones clave.
- ✓ **Explicación en 3 niveles**  
Una frase → evidencia → “cómo llegué aquí” con las consultas.
- ✓ **El humano decide**  
Aprobar, editar o rechazar a un clic; rechazar pide motivo.
- ✓ **Confianza visible**  
Supuestos y nivel de confianza; si duda, lo dice.
- ✓ **Agentes a la vista**  
Se ve el paso en curso mientras analizan.
- ✓ **Lenguaje de negocio**  
Pesos colombianos, fechas claras, cero jerga.
- ✓ **Accesible y responsive**  
Severidad no solo por color; teclado; celular.

# Pantallas del MVP

[⏪ Menú](#)

| Pantalla              | Para quién                  | Qué muestra                                                                    |
|-----------------------|-----------------------------|--------------------------------------------------------------------------------|
| Bandeja de decisiones | Gerente, líderes de proceso | Alertas ordenadas por pesos en riesgo, con urgencia, confianza y acciones      |
| Detalle de alerta     | Líderes de proceso          | Qué pasó, causa con evidencia, acciones propuestas con impacto y aprobación    |
| Chat anclado          | Todos                       | Preguntas sobre la alerta o los datos; respuestas con cifra, gráfico y fuente  |
| Bitácora              | Auditoría, gerencia         | Quién aprobó qué, cuándo, qué ejecutó el agente y con qué resultado            |
| Configuración         | Analista                    | KPIs vigilados, umbrales, responsables y nivel de autonomía por tipo de acción |



# IA responsable: un agente que actúa solo se compra si se puede controlar

| Riesgo                     | Control mínimo exigido                                                                           |
|----------------------------|--------------------------------------------------------------------------------------------------|
| Cifras inventadas          | Todo número viene de una consulta registrada; la respuesta enlaza la consulta que lo produjo     |
| Acción no deseada          | Aprobación humana antes de toda acción externa; lista cerrada de herramientas; modo borrador     |
| Inyección de instrucciones | Datos y documentos son datos, nunca órdenes; el jurado prueba un texto malicioso en una política |
| Acceso indebido a datos    | Usuario de base de datos de solo lectura; roles por área                                         |
| Datos personales           | Enmascarar datos sensibles antes de enviarlos al modelo; Ley 1581 de protección de datos         |
| Falta de trazabilidad      | Bitácora inmutable: alerta, evidencia, propuesta, decisión, acción y resultado                   |
| Degradación del agente     | Set de evaluación con los escenarios sembrados, ejecutado en cada cambio importante              |

Informa

Propone

Ejecuta

Niveles de autonomía por tipo de acción. En el hackatón todo queda en “Propone”; la autonomía plena se gana en producción con historial de aciertos.

[← Volver al menú](#)



# Éxitos...

**Empieza ya a demostrar tus  
capacidades!!**